

Name: Ayan Jafarova

ID: 19824

Microprocessor Systems

Lab Report: Lectures 6-8

[GitHub Repository](#)

Task 6: Using register-level programming and parallel I/O, configure the microcontroller's hardware timer to drive a single-digit 7-segment display. The display should count from 0 to 9, updating once every 0.5 seconds. A push button connected to an external interrupt should pause the counter when pressed and resume it when pressed again. After the display reaches 9, it resets

. Questions to think about:

- How can you be confident that it is counting in precise 0.5-second increments?
- How did you calculate this?
- Does the counter resume exactly where it left off?

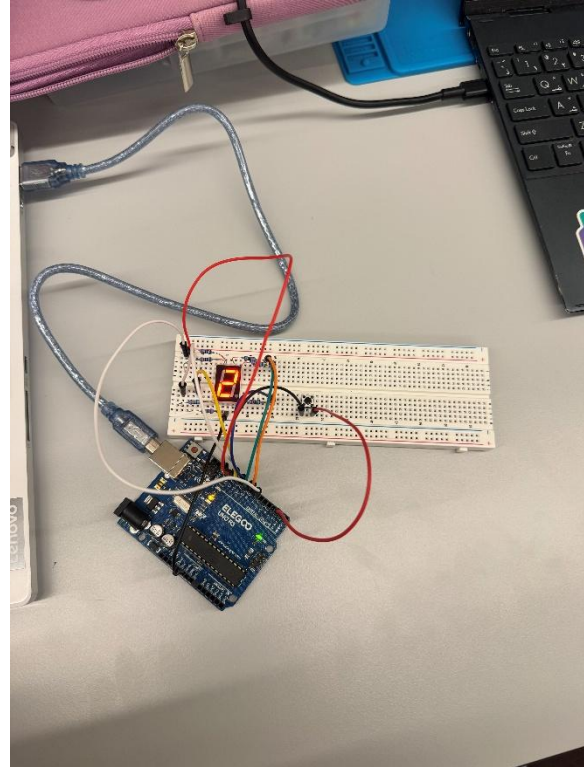
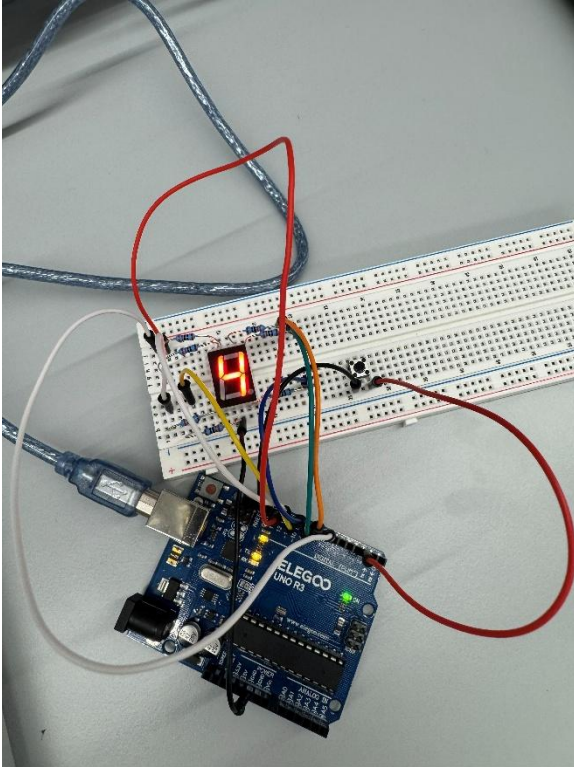
Task Overview

The main goal of this lab was to configure the microcontroller using register-level programming to drive a single-digit 7-segment display, which would increment the value from 0 to 9 updating in 0.5 seconds. Also, a push button needed to be configured as an external interrupt, pressing which would instantly pause the counter and resume on the second press.

Hardware Configuration

The 7-segment display is related to parallel I/O pins, where each segment corresponds to one output bit position.

Segment	Bit Position
A	Bit 0 / PB0
B	Bit 1 / PB1
C	Bit 2 / PB2
D	Bit 3 / PB3
E	Bit 4 / PB4
F	Bit 5 / PB5
G	Bit 6 / PD7



The push button was connected to the pin D2 using the internal pull-up resistor.

Lookup Table for Digits

Digit	Hexadecimal Value	Binary Value
0	0x3F	00111111
1	0x06	00000110
2	0x5B	01011011
3	0x4F	01001111
4	0x66	01100110
5	0x6D	01101101
6	0x7D	01111101
7	0x07	00000111
8	0x7F	01111111
9	0x6F	01101111

Timer Configuration and 0.5-second CTC Increment Calculation

Timer1 was configured in CTC mode (enabled using WGM12). The compare value was calculated by the formula:

$$OCR1A = \frac{CPU\ Frequency}{Prescaler * Interrupt\ Frequency} - 1$$

CPU Frequency = 16 MHz (ATmega328P frequency)

Interrupt Frequency = 0.5 seconds

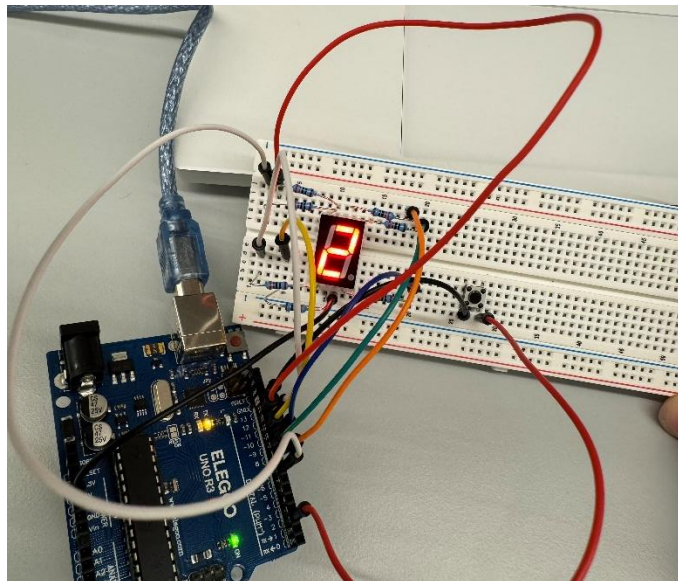
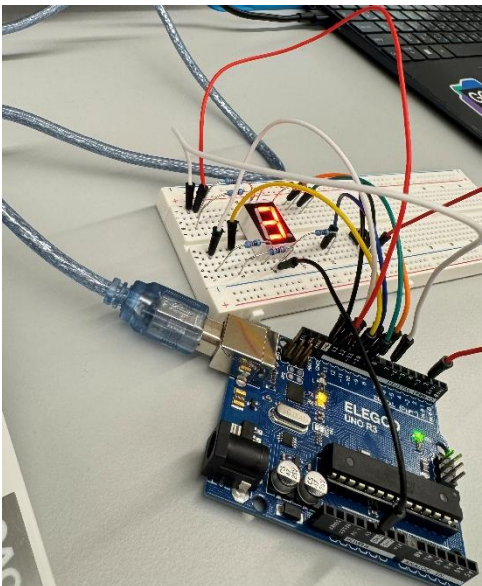
Prescaler = 1024 (Setting bits CS12 and CS10 in TCCR1B)

Dynamic Counter Resume

The external push-button interrupt (ISR) on pin INT0 sets the logic flag (paused), but it does not stop the hardware timer clock behind the process. As a result, the register TCNT1 continues to increment in the background during the pause.

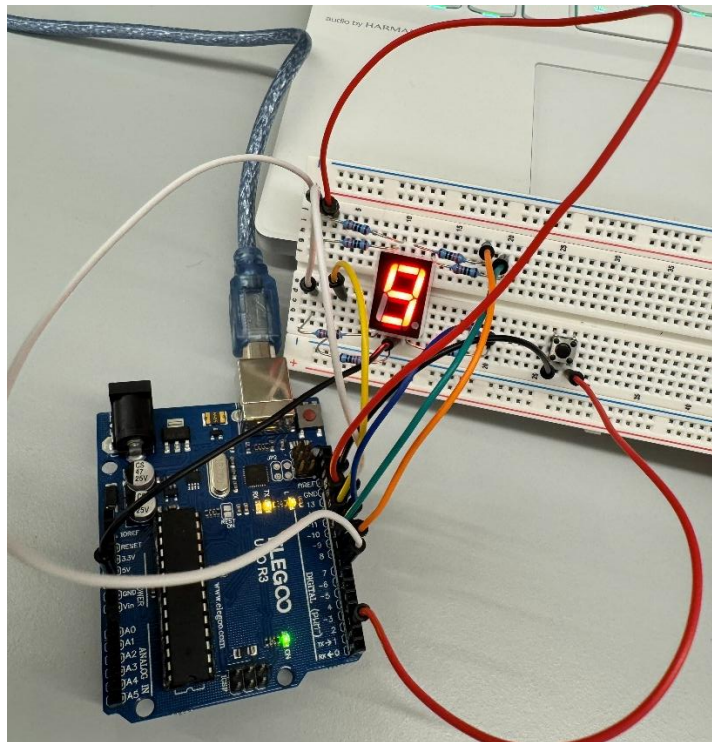
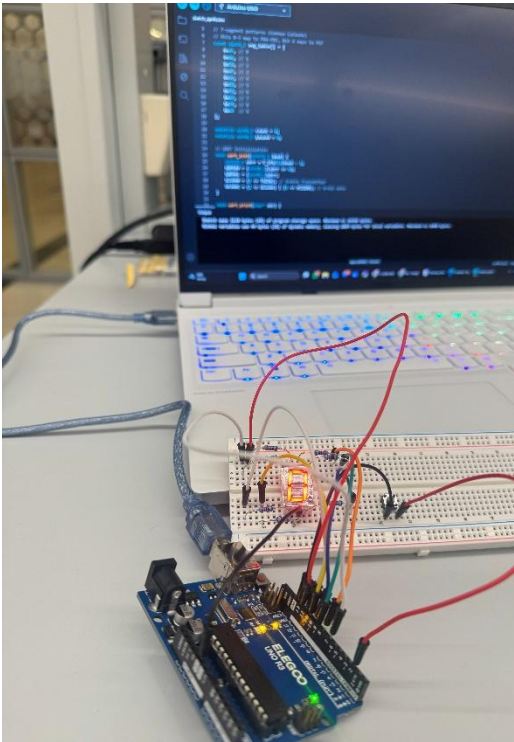
Interrupt Configuration

Register	Function
DDRB	0x3F
DDRD &= ~(1 << DDD2)	Configure INT0 as input
ISCO1 = 1	Falling edge
EIMSK	(1 << INT0)



Debounce Logic

A push button can sometimes rapidly transition for several milliseconds due to physical contact bounce. In order to solve this, we implement a timer-based debounce mechanism in the code. The logic is that when the button interrupt occurs, if the elapsed time is shorter than the debounce ticks, then the interrupt is ignored.



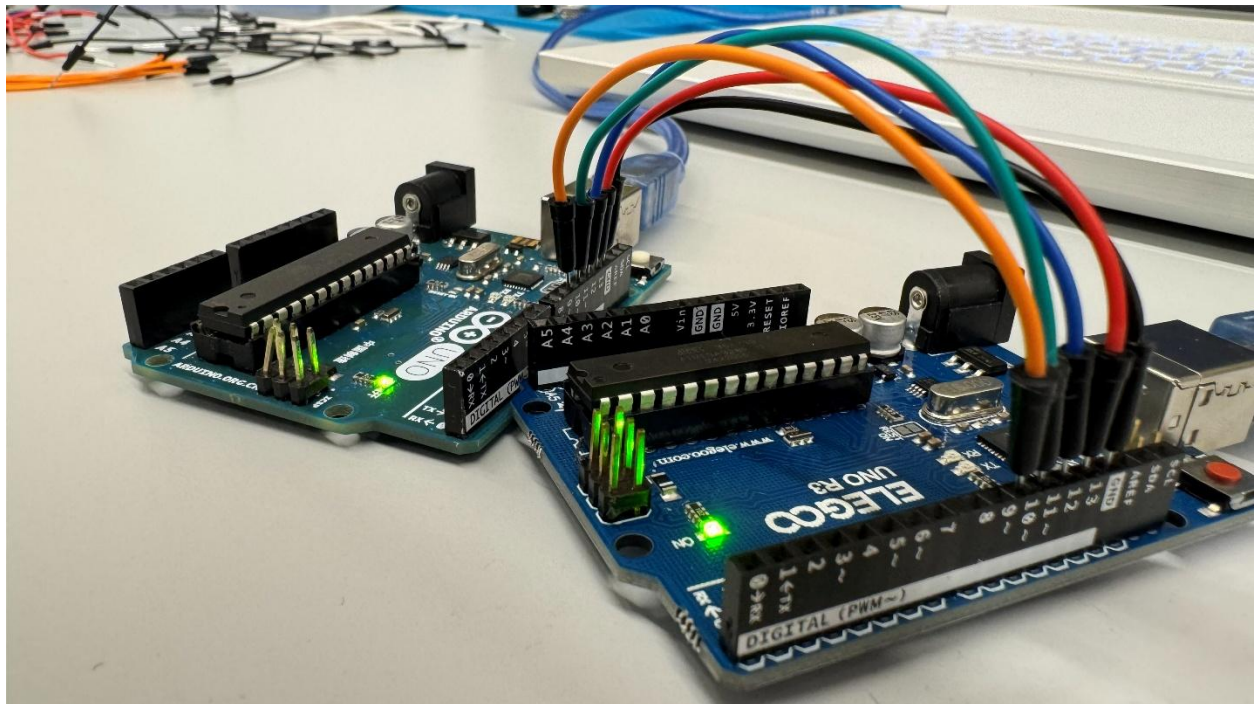
Task 7: Your goal is to use two ATmega328P Arduinos to demonstrate basic SPI communication at the register level. One Arduino must act as the Master and the other as the Slave. The Master must send a single decimal value (0–255) once every second through SPI, and the Slave must correctly receive that value and display it on the Serial Monitor. You must configure all SPI settings using the SPCR, SPSR, SPDR, DDRB, and PORTB registers only.

The Master must transmit a sequence of three specific decimal values every second in the following order: 85 → 170 → 255 (repeat).

Evidence how can you show what is happening with SS, SCK, and MOSI. Draw a timing diagram for one of the decimal values showing how the digital signals (SS, SCLK, MOSI, MISO) change over time.

Task Overview

This task implements the Serial Peripheral Interface (SPI) communication between two Arduino Uno microcontrollers using direct AVR register programming only. The Master manages the data framing and clock, while Slave acts as a passive listener. Master sends three specific values (85 → 170 → 255) every second in a cycle. The Slave will remain listening in the background mode until an incoming byte finishes shifting into its buffer. Next, the SPI interrupt triggers, and Slave reads the ISR data byte by byte from the data register and then transmits the 9600-baud UART configuration to the Serial Monitor.

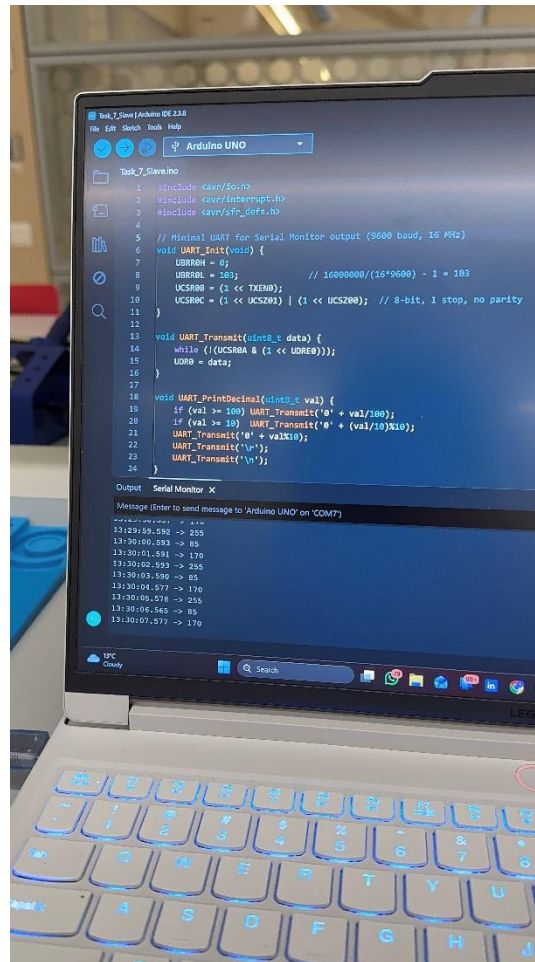


Register and Bit Configurations

Master	Configuration	Slave	Configuration
$\text{DDRB} = (1 \ll \text{PB3}) (1 \ll \text{PB5}) (1 \ll \text{PB2});$	Configures MOSI (PB3), SCK (PB5), and SS (PB2) as outputs to drive the SPI bus lines. Its MISO (PB4) pin is left as an input to receive data from the slave.	$\text{DDRB} = (1 \ll \text{PB4});$	Configures the Slave's MISO (PB4) pin as an output, while its MOSI, SCK and SS pins are set as high-impedance inputs to accept signals from the Master.
$\text{SPCR} = (1 \ll \text{SPE}) (1 \ll \text{MSTR}) (1 \ll \text{SPR0});$	Configures the Master's SPI control registers. SPE register powers on the SPI peripheral hardware and overrides standard GPIO controls on Port B. MSTR register sets the chip as the Master clock controller. SPR0 register selects the prescaler, while leaving the CPOL=0 and CPHA=0 will select SPI Mode 0.	$\text{SPCR} = (1 \ll \text{SPE}) (1 \ll \text{SPIE});$	Sets the Slave's peripheral to slave mode (MSTR = 0). It enables the SPI hardware block (SPE = 1) and turns on the complete-transfer interrupt (SPIE = 1).

Numeric Values in Hexadecimal Representation

Values	Hexadecimal	Binary
85	0x55	0b01010101
170	0xAA	0b10101010
255	0xFF	0b11111111

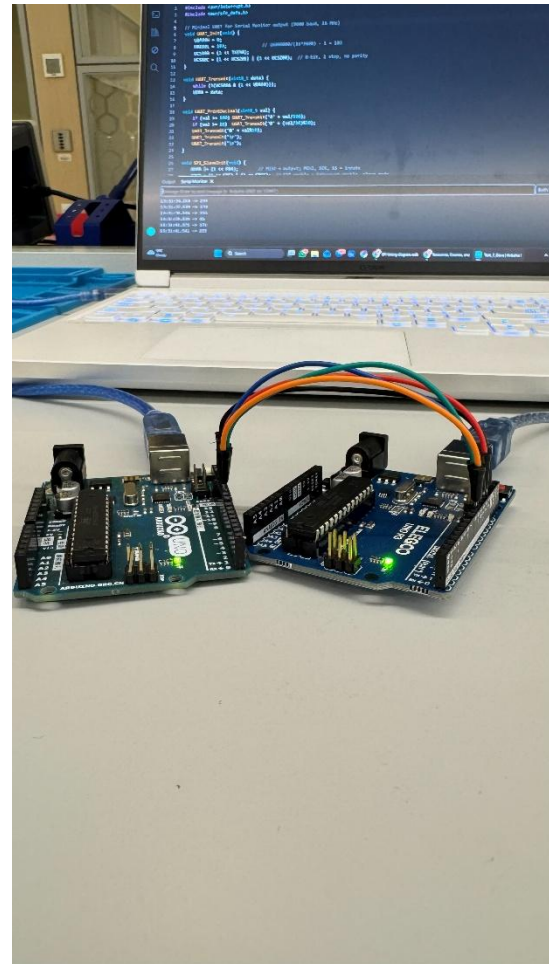
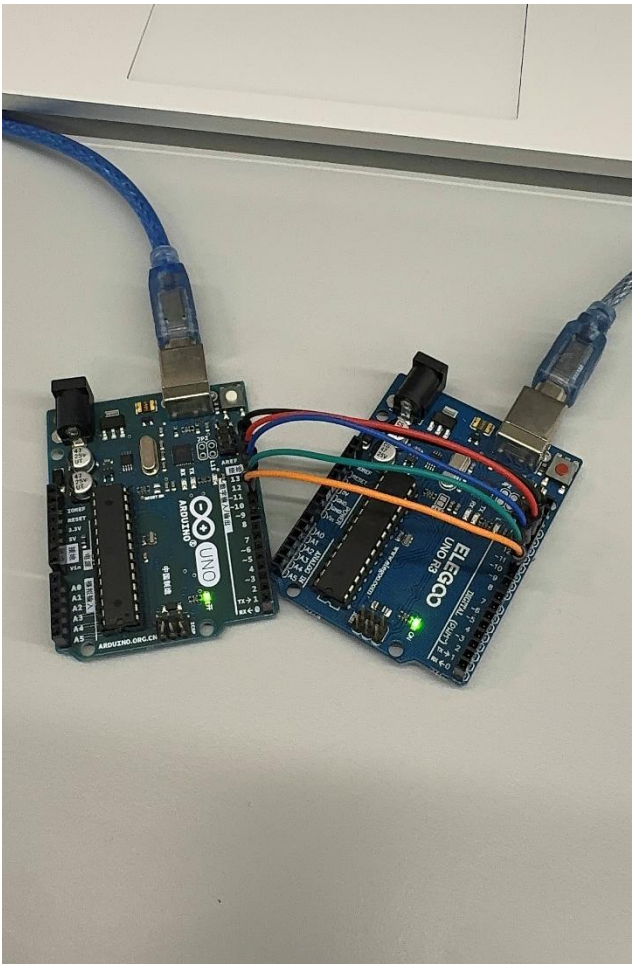


The multi-channel wave capture below isolates the MOSI and SCK of the SPI communication stream and the active transmission data lines during a 1-byte packet frame.



SPI Hardware Connection

Signal	Master Pin	Slave Pin
SS	PB2 / D10	PB2 / D10
MOSI	PB3 / D11	PB3 / D11
MISO	PB4 / D12	PB4 / D12
SCK	PB5 / D13	PB5 / D13
GND	GND rail	GND rail



Task 8: Set up two Arduino boards communicating via I2C (one master, one slave), each with a push button and LED; pressing Button A should send a message to Board B to light its LED, and pressing Button B should send its state back to Board A to light its LED.

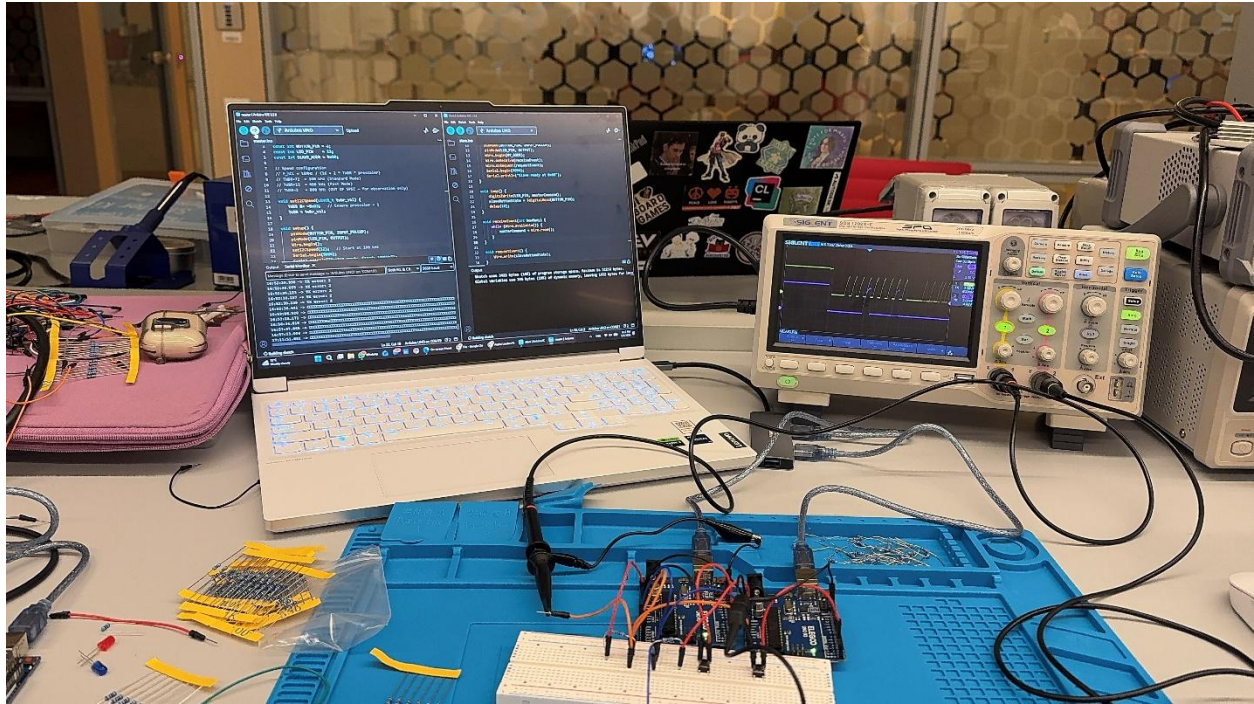
Explain your message protocol design in your report. Then, explore how clock speed affects the I2C signals by modifying the TWI clock registers (TWBR and prescaler bits in TWSR) to test at least two different speeds.

Use an oscilloscope to observe the SDA and SCL waveforms at each speed and document the timing characteristics (bit period, rise/fall times, signal quality, and any ringing or noise).

Discuss how speed changes affect the observed waveforms, any timing violations or signal degradation you encounter, and the practical limits of increasing clock speed on your specific hardware setup → include oscilloscope screenshots and explain what constraints (capacitance, trace length, pull-up resistors, or device slew rates) are likely responsible for any limitations you observe.

Task Overview

The objective of this task is to implement a bidirectional I2C communication link between two Arduino Uno microcontrollers using register-level Two-Wire Interface (TWI) configuration via pins A4 (SDA) and A5 (SCL). Both devices need to use a push button and an LED to interact. Pressing Button A (Master's) sends a message to Board B (Slave) and Board B (Slave) lights its LED. And vice versa, pressing Button B (Slave's) sends a message back and Board A (Master) lights its LED. Two processes are happening: first, the Master reads its local button state and transmits it directly to Slave address (0x08). The Slave processes this incoming command byte inside its TWI interrupt service (TWI_vect), turning its local LED on or off to match the button state. Second, the Master issues an I2C read request to the Slave. The Slave responds by transmitting its local button state back to the Master. The Master receives this state byte and updates its indicator LED accordingly.



Register and Bit Configurations

Register / Code	Purpose
<pre>DDRC &= ~(1 << DDC4) (1 << DDC5); PORTC = (1 << PORTC4) (1 << PORTC5);</pre>	Configures the TWI data (SDA) pin (PC4) and clock (SCL) pin (PC5) as input and enables internal pull-ups on those pins. This allows for the use of the open-drain I2C protocol, where the lines are pulled low by the device, but brought high by the pull-ups.
<pre>PRR &= ~(1 << PRTWI);</pre>	Sets the Power Reduction TWI bit in the Power Reduction Register. This makes sure that the TWI hardware module gets its clock signal and becomes completely enabled before data transfer starts.
<pre>TWCR = (1 << TWINT) (1 << TWSTA) (1 << TWEN);</pre>	Issues the START condition for the I2C protocol. By writing a 1 to TWINT, the interrupt flag is cleared for the last operation, TWSTA sends the START condition on the bus, and TWEN enables the TWI hardware module.
<pre>TWAR = (address << 1);</pre>	Loads the address of the slave device into the TWI Address register. The address needs to be left-shifted once because the least significant bit of the address is reserved by the hardware for the direction flag.

$TWCN = (1 \ll TWEN) (1 \ll TWEA) (1 \ll TWIE);$	Sets up ATmega328P as an active I2C slave. TWEN sets up the TWI module for communication, TWEA automatically generates the acknowledgement of the address by the device, and TWIE sets up the TWI module for interrupt mode using TWI_vect interrupt vector.
--	--

TWI Clock Configuration

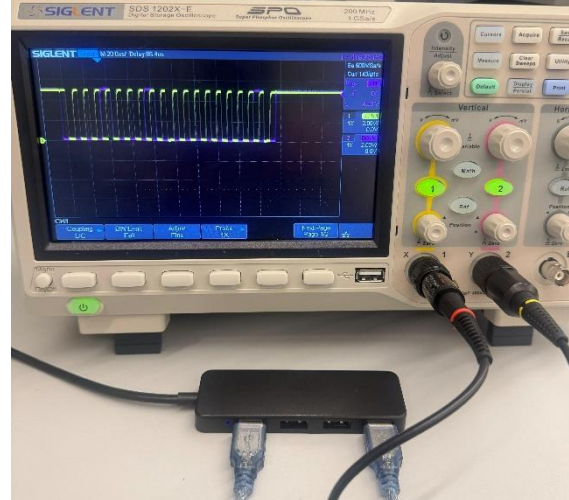
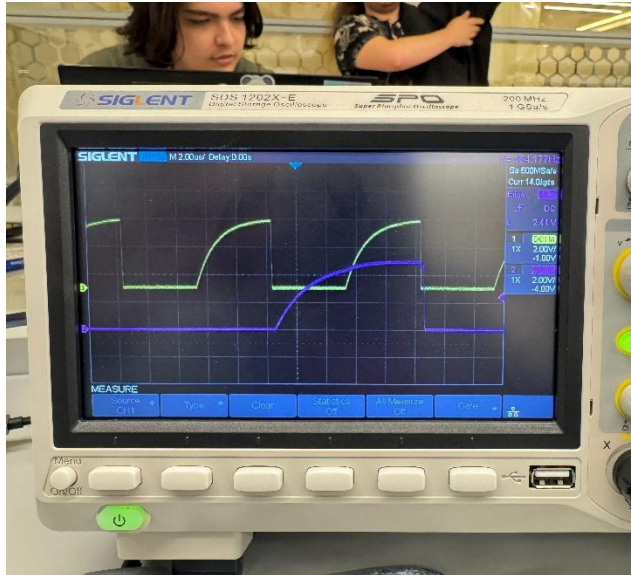
The I2C clock frequency is calculated with the formula given below. We needed to implement and test two speed modes, Normal and Fast (100 kHz and 400kHz). The TWBR (TWI Bit Rate Register) values were 72 and 12, respectively, with prescaler 1 for both speeds.

The **Two-Wire Interface (TWI)** on ATmega328P sets the SCL clock using:

$$f_{SCL} = \frac{f_{CPU}}{16 + 2 \cdot TWBR \cdot 4^{prescaler}}$$

Oscilloscope Wave Measurements

To establish a standard 100 kHz clock speed, the bit rate register value is calculated as $\rightarrow 16 \text{ MHz} / (16 + 2 (TWBR * 1)) = 100 \text{ kHz} \rightarrow TWBR = 72$. Waveforms on both the SCL and SDA lines show excellent signal integrity. The rising and falling edges are sharp and vertical, with negligible voltage undershoot or ringing. The signal easily stays within safe noise margins. To shift the bus up to a faster 400 kHz clock speed, the TWBR value is adjusted $\rightarrow TWBR = 12$. The waveforms show significant signal degradation. The rising edges are heavily rounded, stretching out into an arch shape. This happens because the weak internal pull-up resistors cannot clear the bus capacitance fast enough during the brief clock window, which can cause communication errors.



The RC Time Constant and Open-Drain Outputs

Degradation and rounding seen during rising edges at higher clock speeds are a direct result of the effect caused by the interconnection between the pull-up resistors and the dependent capacitance of the bus, using the RC time constant formula. Due to the open-drain configuration used by the I2C pins, active driving of the bus lines can only be done when the level is low. Bringing the line to high is purely dependent on the pull-up resistor.

Rising edge follows:

$$\tau = R \cdot C \rightarrow (RC \text{ time constant})$$

To go faster:

R must be smaller (stronger pull-up)

C must be lower (shorter traces, fewer devices)

R = pull-up resistor

C = bus capacitance

Constraints of Physical System and Possible Solutions Implemented

In order to eliminate this performance limitation and ensure that we have clean and square waveforms at such high speeds, we can decrease the RC time constant by making changes in our hardware system, such as decreasing resistance (R) and decreasing capacitance (C). We can disable the weak internal 10k Ω pull-up resistors through software and add more powerful external discrete pull-up resistors (2.2k Ω or 4.7k Ω) to both SDA and SCL lines. This would increase the pull-up current so that we can charge our bus capacitance much faster. We can minimize dependent capacitance by reducing the length of wires, bringing the microcontrollers closer to the breadboard, and using low-capacitance shielded cables.

